



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 11

Team No.: 7

Project Title: Linux Inter-Process Communication Using Pipes and FIFOs

Team Members

Roll Number	Student Name	Signature
2520090181	Himasree	
2520080063	Lalitha	
2520090050	Sravya	

1. Abstract

Inter-Process Communication (IPC) is an important concept in operating systems that enables multiple processes to exchange information and coordinate their activities. This project, “Linux Inter-Process Communication Using Pipes and FIFOs,” focuses on understanding and implementing two fundamental IPC mechanisms available in Linux: anonymous pipes and named pipes (FIFOs). Pipes provide a unidirectional communication channel that is commonly used between related processes, such as a parent and child process. FIFOs extend this concept by allowing communication between unrelated processes through a special file created in the file system.

The project demonstrates the complete process of communication, including creating communication channels, sending data from one process to another, reading the transmitted data, and properly closing the communication resources. Linux system calls such as pipe(), mkfifo(), fork(), read(), write(), and close() are used to implement these mechanisms. The project also demonstrates synchronization between processes to ensure that data is transferred correctly without conflicts or loss.

Through this implementation, the project provides practical knowledge of process creation, inter-process data exchange, synchronization, and file-based communication in a Linux environment. The performance and behaviour of pipes and FIFOs can also be observed under different communication scenarios. Overall, this project helps develop a strong understanding of how Linux manages communication between concurrent processes and demonstrates how IPC mechanisms can be used to build efficient and coordinated multi-process applications. It serves as a practical foundation for studying advanced Linux operating-system concepts such as process synchronization, concurrent programming, and communication mechanisms.

2. Problem Statement

In a Linux operating system, multiple processes often need to exchange data and coordinate their execution. Since processes have separate memory spaces, they cannot directly access each other's data. This creates a need for reliable Inter-Process Communication (IPC) mechanisms. Without proper communication and synchronization, processes may experience data loss, incorrect data transfer, or synchronization problems.

This project addresses the problem of establishing reliable communication between processes using Pipes and FIFOs. Anonymous pipes are used to facilitate communication between related processes, while named pipes (FIFOs) enable communication between unrelated processes. The system demonstrates how processes can transfer data, synchronize their operations, and manage communication resources using Linux system calls such as `pipe()`, `mkfifo()`, `fork()`, `read()`, and `write()`.

This problem is directly relevant to Operating Systems because process communication, synchronization, and resource management are fundamental OS responsibilities. It is also relevant to Systems Programming because the implementation uses low-level Linux system calls to directly interact with processes and communication resources. The project therefore provides practical understanding of how Linux manages communication between concurrent processes.

3. Objectives

1. To implement inter-process communication between related processes using anonymous Pipes in Linux.
2. To implement communication between unrelated processes using Named Pipes (FIFOs).
3. To demonstrate reliable data transfer, synchronization, and coordination between processes using Linux system calls.
4. To understand and apply Linux IPC concepts and system calls such as `pipe()`, `mkfifo()`, `fork()`, `read()`, `write()`, and `close()`.

4. Proposed Methodology

The project will be designed and implemented in a Linux environment using the C programming language and Linux system calls. The system will demonstrate two IPC mechanisms: anonymous Pipes and Named Pipes (FIFOs).

First, an anonymous Pipe will be created using the `pipe()` system call. A parent process will create a child process using `fork()`. The processes will communicate by writing data into the Pipe and reading the data from the other end. Appropriate file descriptors will be closed to ensure proper synchronization and communication.

Next, a Named Pipe (FIFO) will be created using the `mkfifo()` system call. Separate processes will use the FIFO as a communication channel, allowing an independent writer process to send data and a reader process to receive it. The `open()`, `read()`, `write()`, and `close()` system calls will be used for data transfer and resource management.

The overall flow will be: Process Creation → IPC Channel Creation → Data Writing → Data Reading → Process Synchronization → Resource Closure. The output will demonstrate successful communication and data transfer between processes.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
Pipe()	Creates an anonymous pipe for communication between related processes.
Fork()	Creates a child process to demonstrate communication between parent and child.
mkfifo()	Creates a named pipe (FIFO) for communication between unrelated processes.
read() / write()	Used to receive and transmit data through the Pipe or FIFO.

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520090181	Himasree	Design and implementation of anonymous Pipe communication using pipe(), fork(), read(), and write().
25200080063	Lalitha	Design and implementation of Named Pipe (FIFO) communication using mkfifo(), open(), read(), and write().
2520090050	Sravya	Integration and testing of both IPC mechanisms, process synchronization, error handling, output analysis, and documentation.

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating environment for implementing and testing the IPC system
C	Programming language used to implement the IPC mechanisms
GCC Compiler	Compiles and builds the source code
Linux Terminal	Used to execute programs and test Pipes and FIFOs
POSIX APIs	Provides standard interfaces for process and IPC operations

8. Expected Outcome

The project is expected to successfully demonstrate reliable Inter-Process Communication in a Linux environment using anonymous Pipes and Named Pipes (FIFOs). The system should allow related processes to exchange data through Pipes and unrelated processes to communicate through FIFOs.

The expected output will show the creation of communication channels, successful transmission and reception of messages, and proper synchronization between processes. The system should also handle basic errors and ensure that communication resources are properly closed after use.

The project will provide a practical understanding of process communication, synchronization, Linux system calls, and resource management. It will demonstrate how Pipes and FIFOs can be used to establish efficient communication between concurrent processes in Linux.

9. GitHub Repository

GitHub Repository Link: <https://github.com/lalitha-0890/OS-Project>

Repository Created: Yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Dasari Alekhya .

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____